

The Bottom Panel (Viz-Dock): Implementation and Workspace Sync

How Seven Tools Share One Dock, and How Each Links Back to the Blocks

Research note — Block-MiniJava

2026-07-20

Abstract

The bottom panel — `<section id="viz-dock">` (`src/index.html:243--327`), internally called the *viz-dock* — hosts seven tools behind one tab strip: **Problems**, **Output**, **Call-by-Structure**, **Call-by-Value**, **CESK**, **A vs B**, and **Rewrite**. All seven share one shell (`src/core/ui/visualizationPanel.ts`) for opening, closing, resizing, and tab-switching, but they fall into three genuinely different *synchronization regimes* with the main Blockly workspace: two tools highlight the live, original blocks; three tools render an independent *copy* of a block subtree in their own private workspace and never touch the original; and one tool is not block-based at all. This note explains the shared shell first, then each regime with the exact functions that implement it.

Contents

1	The shared shell	1
1.1	The info line and passive vs. active tools	2
2	The three sync regimes	3
2.1	Live highlight: Problems	3
2.2	Live highlight: CESK and A vs B	4
2.3	Independent copy: Call-by-Structure and Call-by-Value	5
2.4	Independent copy: Rewrite	6
2.5	Text only: Output	6
3	Full tab inventory	7
4	HTML wiring	7
	Note on scope	8

1 The shared shell

`src/core/ui/visualizationPanel.ts` (313 lines) owns everything about the dock that is common to all seven tools: which one is active, whether the dock is open, its height, and where each tool's DOM host lives. It does not itself know how to run a machine or check types — those are injected as callbacks or driven by other modules that this one only calls into.

Symbol	Role
VizKind	visualizationPanel.ts:15 — the closed union ReductionKind 'machine' 'compare' 'subst' 'problems' 'output', i.e. 'structure' 'value' 'machine' 'compare' 'subst' 'problems' 'output' — one tag per tab.
ALL_KINDS	visualizationPanel.ts:18 — the tab order as rendered: problems, output, structure, value, machine, compare, subst.
TITLE	visualizationPanel.ts:19--27 — one human-readable description per kind, shown in #viz-dock-info (§1.1).
hostOf(kind)	/ visualizationPanel.ts:62--72 — look up .viz-host[data-kind=kind] /
tabOf(kind)	
	.viz-tab[data-kind=kind] in the DOM; every other function addresses a tab/host pair only through these two, so the seven tools never need seven special cases.
setActive(kind)	visualizationPanel.ts:128--146 — the single tab-switch function: toggles data-active/hidden/aria-* on every host and tab, calls setStepperTabVisible / setCompareTabVisible / setSubstTabVisible so the newly-hidden or newly-shown stepper can pause its Play loop or redraw its SVG overlay (companion note cesk-machine-and-stepper.tex, §4.1), persists the choice to localStorage, and resizes the active tab's own Blockly instance.
openBottomTool(kind)	visualizationPanel.ts:148--151 — setActive(kind) then force-opens the dock; the single entry point every "jump to a specific bottom tool" call site uses (Problems from the status bar, §2.1; Run opening Output, app.ts:340).
setVizOpen(open)	visualizationPanel.ts:157--173 — shows/hides the whole dock (dock().dataset.open), clears "maximized" state on close, persists to localStorage, and calls the shell's onLayoutChange callback (installed by initVisualizationPanel, wired from app.ts's layout coordinator — see ui-design-and-layout.tex).
initResizer()	visualizationPanel.ts:216--251 — pointer-drag height resize on #viz-resizer, plus ArrowUp/ArrowDown keyboard resize (l.245–250); persists height to localStorage.
initVisualizationPanel(layoutChange)	visualizationPanel.ts:253--313 — called once from app.ts; wires every tab's click/keyboard-arrow navigation (l.255–273), the toolbar's Re-run/Arrange/Collapse/Maximize buttons, restores the persisted height/open/active-tab state, and installs the resizer.

1.1 The info line and passive vs. active tools

updateInfo() (visualizationPanel.ts:111--118) writes TITLE[active] into #viz-dock-info, appending the currently-visualized block's type for Structure/Value. updateToolActions() (l.120–126) hides the **Re-run** button for the two tools that have nothing to re-run (problems,

`output` — both are "passive": they mirror state pushed into them rather than owning a run of their own) and hides **Arrange** for every tool except Structure/Value (only they lay out a tree of copied blocks that benefits from re-arranging).

2 The three sync regimes

Every artifact rendered in the bottom panel is, in principle, a candidate for "click it to jump to the block it came from." Which workspace it jumps *to* is what separates the three regimes below.

Regime	Tabs	Sync target
Live highlight	Problems, CESH, A vs B	The <i>original, live</i> main workspace — the same <code>Blockly.WorkspaceSvg</code> the user is editing. Clicking an artifact centers and selects the real block; a running machine continuously highlights its current focus block there too.
Independent copy	Call-by-Structure, Call-by-Value, Rewrite	A <i>separate</i> Blockly workspace injected inside the dock, showing a freshly rendered copy of a block subtree. Clicking inside these tabs never touches the main workspace, because the tree on screen is not the same object as the blocks it was copied from.
Text only	Output	Not block-based at all — a plain <code><pre></code> console; there is nothing to click-to-locate.

2.1 Live highlight: Problems

`src/core/ui/typeDiagnostics.ts` (104 lines) is the smallest and clearest example of the pattern every other live-highlight tool follows.

Function	Role
<code>refreshTypeDiagnostics(workspace)</code>	<code>typeDiagnostics.ts:99--104</code> — the one exported entry point: runs <code>checkWorkspace</code> (the type checker, documented in <code>typing-and-evaluation-rules.tex</code>), then refreshes <i>both</i> surfaces from the same diagnostic list, so they cannot disagree. Called from the workspace's debounced type-check timer (<code>app.ts</code>) and from <code>showProblems()</code> (<code>app.ts:480--483</code>).
<code>applyBlockWarnings(workspace,diags)</code>	<code>typeDiagnostics.ts:23--34</code> — surface #1: a warning icon <i>on the block itself</i> , via <code>block.setWarningText(..., WARNING_OWNER_ID)</code> . Tracks <code>markedBlockIds</code> (1.11) across calls so a block whose diagnostic disappeared gets its warning explicitly cleared (1.25–29) rather than accumulating stale icons.

Function	Role
<code>renderProblemsPanel(workspace,diags)</code>	<code>typeDiagnostics.ts:76--96</code> — surface #2: the Problems list itself (<code>#bottom-problems-list</code>) plus two badge counters (<code>#bottom-problems-count</code> on the tab, <code>#status-problems-count</code> in the status bar (see <code>ui-design-and-layout.tex</code>), both toggling a <code>has-errors</code> class when at least one diagnostic is an error rather than a warning.
<code>renderProblemList(list,workspace,diags)</code>	<code>typeDiagnostics.ts:46--74</code> — builds one <code>button.problem-item</code> per diagnostic, sorted errors-first (1.56–58); each button’s click handler is exactly <code>locateBlock(workspace, diag.blockId)</code> .
<code>locateBlock(workspace,blockId)</code>	<code>typeDiagnostics.ts:37--44</code> — the canonical click-to-locate implementation , reused verbatim by the Typing tab (<code>typingPanel.ts:26--30</code> , per <code>typing-code-outline-tabs.tex</code>): <code>workspace.getBlockById(blockId)</code> , then <code>workspace.centerOnBlock(blockId)</code> , then <code>Blockly.common.setSelected(block)</code> (which, unlike <code>BlockSvg.select()</code> , first clears whatever was previously selected), and finally dispatches a <code>'bmj:problem-located'</code> <code>CustomEvent</code> on window so other listeners (if any) can react to a problem having been jumped to.

2.2 Live highlight: CESK and A vs B

The two machine steppers extend the same idea from “click once to locate” to “continuously highlight the machine’s current focus, and make *every* rendered value/frame/kontinuation entry individually clickable.” Both are documented in full implementation depth in `cesk-machine-and-stepper.tex`; this section gives only the sync mechanism, for completeness of the bottom-panel inventory.

Tab	Function	What it does
CESK	<code>locateProvenance(blockId)</code> — <code>stepperPanel.ts:90--100</code>	Click-to-locate on any <code>.stepper-provenance</code> element (control text, a frame’s method name, a heap box’s title, a kontinuation entry) — identical three-call idiom to <code>locateBlock</code> above.
CESK	<code>setHighlight(blockId)</code> — <code>stepperPanel.ts:119--133</code>	<i>Continuous</i> highlight: called from <code>renderAll()</code> (1.488) after every step with <code>current.focusBlockId</code> , so the program surface always shows exactly where the machine currently is, independent of any click. Un-highlights the previous focus block first (1.122–124).

Tab	Function	What it does
A vs B	<code>setHighlights(blockId s) — comparePanel.ts:71--83</code>	The two-machine analogue: both models' current focus blocks are highlighted <i>simultaneously</i> on the one shared main workspace (<code>getWorkspace()</code> at l.72 is the same closure passed to <code>stepperPanel</code>), so a user watching the lockstep view sees both machines' positions in the same program surface at once.

Both stepper panels take a `GetWorkspace` closure returning the *live* main `Blockly.WorkspaceSvg` at `init` time (`stepperPanel.ts:857`, `comparePanel.ts`'s equivalent) — this is what makes "live highlight" possible at all: the machine (`injectMachine(workspace, model)`) is built by reading the real workspace's blocks, so the `Blockly.Block` references a `MachineState`'s `control/frames/heap` carry (as `blockId` strings, resolved back via `getBlockById`) always resolve in that same live workspace.

2.3 Independent copy: Call-by-Structure and Call-by-Value

These two tabs are invoked differently from everything else in the dock: not from a tab click, but from a **right-click context menu** on a specific method-call block in the main workspace.

Symbol	Role
<code>registerMiniJavaContextMenus()</code>	<code>contextMenus.ts:17--62</code> — registers two <code>Blockly.ContextMenuRegistry</code> items, <code>miniJavaVizStructure</code> and <code>miniJavaVizValue</code> , scoped to <code>ScopeType.BLOCK</code> and shown only when the right-clicked block is <code>mj_expr_method_call</code> (<code>contextMenus.ts:28,38</code>); each callback calls <code>openVisualization(kind, scope.block)</code> .
<code>openVisualization(kind, block)</code>	<code>visualizationPanel.ts:192--197</code> — stores the seed block in <code>views[kind].block</code> , activates that tab, opens the dock, and calls <code>renderView(kind)</code> .
<code>renderView(kind)</code>	<code>visualizationPanel.ts:175--190</code> — clears (or lazily creates via <code>ensureWorkspace</code> , l.95–98) a <i>brand-new</i> <code>Blockly.WorkspaceSvg</code> injected into that tab's own <code>.viz-host</code> (<code>injectVisualizationWorkspace</code> , l.78–93, with its own zoom/grid/theme settings), then calls <code>renderMiniJavaReduction</code> or <code>renderCopiedReductionRoot</code> (<code>core/semantics/minijavaReduction.ts</code>) to populate it with a <i>copy</i> of the reduction tree rooted at the seed block.

Why no click-to-locate back to the main workspace: the blocks inside the Structure/Value workspace are independent copies (fresh block ids, via `Blockly.serialization.blocks-` style duplication inside `minijavaReduction.ts`) — this is a direct visual consequence of the

substitution semantics these tabs illustrate (`typing-and-evaluation-rules.tex`, §4.3): each reduction step is rendered as its own *self-contained* block tree, and two occurrences of a substituted value are, by the structure-preservation invariant, never the same node. There is accordingly no single “the” main-workspace block a click inside this copy could unambiguously jump back to. `disposeVizWorkspaces()` (`visualizationPanel.ts:199--214`) tears down and, if that tab was open, re-renders these private workspaces on a theme switch, confirming they are wholly owned by the dock and carry no reference back into the main workspace’s block set.

2.4 Independent copy: Rewrite

The Rewrite (substitution stepper) tab, `src/core/ui/substPanel.ts` (295 lines), follows the same “own private workspace” pattern as §2.3, but is entered from the tab strip directly (like CESK) rather than from a context menu, and highlights within that private workspace continuously as it steps.

Symbol	Role
<code>getWorkspace</code> (main)	<code>substPanel.ts:32</code> — used <i>only</i> to seed the initial tree (from the <code>println</code> argument in <code>main</code> , per <code>typing-and-evaluation-rules.tex</code> §4.3.1) and to detect staleness on a live edit (<code>attachWorkspaceListener</code> , <code>substPanel.ts:97--118</code> — same <code>BLOCK_CREATE/DELETE/CHANGE/MOVE</code> listener shape as every other stepper). It is never the target of a click-to-locate.
own highlight (redex)	<code>substPanel.ts:176</code> — <code>host.getBlockById(current.redexId).setHighlighted(true)</code> , where <code>host</code> is the tab’s own private workspace (populated by re-rendering <code>current.tree</code> — a <code>TreeNode</code> , not a <code>Blockly.Block</code> — into fresh blocks each step). The highlighted block is always inside <i>that</i> tree, never the main program’s.

This is the clearest illustration of the regime split in this note: CESK and A vs B evaluate the *actual program blocks by reference* (`Blockly.Block` objects looked up live by id), so highlighting them *is* highlighting the main workspace; Structure/Value/Rewrite evaluate a *copied, self-contained syntax tree* (the substitution semantics’ own state representation, per the companion evaluation-rules note), so their highlight necessarily targets their own private copy.

2.5 Text only: Output

`src/core/ui/programConsole.ts` (50 lines) is the simplest tool in the dock and syncs with nothing block-related at all — its “source of truth” is a plain string log, not a workspace.

Function	Role
mirrorProgramOutput (source,lines,note)	programConsole.ts:24--40 — overwrites #bottom-program-output with [source] plus the joined println lines, flashing is-console-changed when the body actually changed (1.31–34). Whichever tool produced output most recently owns the console: Run (app.ts:340--351), the CESK stepper (stepperPanel.ts:477), and A vs B all call this same function, so the Output tab is a shared monitor, not one tool’s private log.
appendConsoleLog(message)	programConsole.ts:43--50 — appends one line (used for editor-action notices like export/clipboard) without replacing the mirrored run output.

3 Full tab inventory

Tab	data-kind	Sync regime	Owning module	
Problems	problems	Live highlight	typeDiagnostics.ts	
Output	output	Text only	programConsole.ts	
Call-by-Structure	structure	Independent copy	visualizationPanel.ts	+
Call-by-Value	value	Independent copy	minijavaReduction.ts	
			visualizationPanel.ts	+
			minijavaReduction.ts	
CESK	machine	Live highlight	stepperPanel.ts	
A vs B	compare	Live highlight	comparePanel.ts	
Rewrite	subst	Independent copy	substPanel.ts	

4 HTML wiring

Element	Location
#viz-dock	index.html:243 — the dock root; data-open toggled by setVizOpen.
.viz-dock-tabs	index.html:247--268 — the seven button.viz-tab[data-kind] elements, in ALL_KINDS order.
#viz-dock-info	index.html:270 — the per-tab description line (§1.1).
.viz-dock-tools	index.html:271--276 — Re-run (#viz-rerun), Arrange (#viz-arrange), Maximize (#viz-maximize), Hide (#viz-collapse).

Element	Location
<code>.viz-host[data-kind]</code>	<code>index.html:279--328</code> — seven host containers, one per tab; <code>problems/output</code> are <code>.bottom-passive-host</code> (plain content, no injected Blockly workspace); <code>structure/value/machine/compare/subst</code> each host either an injected reduction workspace or the CESK/Compare/Rewrite stepper layout (documented in <code>cesk-machine-and-stepper.tex</code> and <code>typing-and-evaluation-rules.tex</code> respectively).
<code>#viz-resizer</code>	<code>index.html:244</code> — the drag handle for dock height (§1, <code>initResizer</code>).

Note on scope

This note covers the bottom panel’s *shell* (tab switching, open/close, resize) and its *synchronization* with the workspace. The CESK and A vs B tabs’ machine semantics and internal rendering (Control/Environment/Store/Kontinuation columns, SVG heap arrows, garbage collection) are covered in `cesk-machine-and-stepper.tex` and `garbage-collection.tex`. The Rewrite tab’s substitution semantics are covered in `typing-and-evaluation-rules.tex`. The three inspector-column tabs (Editable Code, Typing, Outline) are a separate panel, covered in `typing-code-outline-tabs.tex`. The workbench shell the dock sits inside (activity bar, perspectives, responsive layout) is covered in `ui-design-and-layout.tex`.